

A TinyML IoT Intrusion Detection System with Human-Readable XAI and Lightweight Drift Monitoring for MCUs

Monish Saha

*Department of Computer Science
American International University-Bangladesh
Dhaka, Bangladesh
21-45573-3@student.aiub.edu*

Oishe Das

*Department of Computer Science
American International University-Bangladesh
Dhaka, Bangladesh
22-49340-3@student.aiub.edu*

Anik Sarker Rudra

*Department of Computer Science
American International University-Bangladesh
Dhaka, Bangladesh
22-48296-3@student.aiub.edu*

Md. Towfiq Bin Hasan

*Department of Computer Science
American International University-Bangladesh
Dhaka, Bangladesh
22-48732-3@student.aiub.edu*

Kamruddin Nur

*Department of Computer Science
American International University-Bangladesh
Dhaka, Bangladesh
kamruddin@aiub.edu*

Abstract—The rapid growth of Internet of Things (IoT) deployments has increased the requirement for real-time intrusion detection at the network edge, where defenses are restricted by latency, bandwidth, and security concerns associated with centralized cloud-based defenses. Contemporary IoT intrusion detection systems (IDS) typically use computationally expensive models that emphasize prediction accuracy or use simplified methods to ensure the TinyML capability, which lacks interpretability and deployment stability. This study creates a TinyML-focused IoT IDS for the Raspberry Pi Pico 2 W Using a lightweight compact neural network trained on a reduced multi-class CICIoT2023 dataset. The proposed pipeline performs well in TinyML-based feature engineering, hybrid feature ranking to determine 24 features, training only preprocessing, multi-metric drift monitoring, and compact multilayer perceptron training combined with human-readable feature identification. The experimental evaluation presents numerous TensorFlow Lite deployment modifications with restrictive microcontroller unit (MCU) parameters. The system reduces the model size to 19.37 KB while achieving 99.89% accuracy and 99.91% macro-F1. The framework exhibits accurate, compact, understandable, and drift-aware intrusion detection, which is appropriate for real-world microcontroller deployments.

Index Terms—edge computing, internet of things (IoT), intrusion detection systems (IDS), lightweight neural networks, microcontrollers (MCU), TinyML

I. INTRODUCTION

The Internet of Things is growing rapidly and changing areas as healthcare, transportation, homes, manufacturing, and cyber-physical systems. However, because of the limited resources of these IoT devices, they become easy targets for

attacks such as botnets, denial-of-service, and malicious traffic. Intrusion detection systems are crucial for security to solve this issue. A realistic large-scale IoT attack environment is represented in the CICIoT2023 dataset. [1].

To detect anomalies in network traffic data, most traditional IoT intrusion detection systems depend on cloud servers, central systems, or powerful edge devices. However, when local processing is required, they are not suitable for latency-sensitive, bandwidth-limited, and privacy-critical IoT environments. Prior studies have used lightweight deep learning methods such as DL-BiLSTM, IPCA, and dynamic quantization [2], along with Random Forest models tuned on CICIoT2023 [3]. The accuracy is improved using decision trees, and ensemble methods [4]. A recent comparative study found that traditional machine learning models perform effectively in multiple IoT device detection environments [5]. However, MCU deployment is difficult because of its high computational cost. TinyML enables on-device cyberattack detection.

TinyML has shown promising results for intrusion detection in embedded microcontrollers. A multilayer perceptron, a small standalone model, has also performed well in these environments [6]. Explainable AI has become important in IDS, where SHAP-based methods are used for interpretation [7, 8]. Concept drift localization has also been explored in IoT datasets [9]. However, most existing methods still struggle to balance accuracy, explainability, and efficiency on MCUs.

The key contributions of this study are as follows.

- 1) Design of IDS architecture based on the Internet of

Things (IoT) using TinyML-based feature extraction, preprocessing through data-only training, and combination of features to design 24 new CICIOT2023 features.

- 2) Design a compact neural network achieving accuracy of 99.89%, macro F1 score of 99.91%, while minimizing TFLite size.
- 3) Deployed on Raspberry Pi Pico 2 W with readable feature outputs and concept-drift detection. The remaining sections of this paper is organized as follows.

The remainder of this paper is structured as follows. Section II presents a literature review of previous studies on IoT intrusion detection systems. Section III elaborates on the proposed approach, including feature selection and preprocessing. Section IV provides the performance analysis of our approach. Section V discusses the limitations of our study and future works. Finally, Section VI concludes the study.

II. RELATED WORKS

The ability to learn complex attack patterns from network traffic makes machine learning and deep learning models widely used in IoT intrusion detection systems. Currently, there is increasing attention towards TinyML human-readable explanations and concept-drift-aware methods for building trustworthy resource-constrained IoT environments.

Wang et al. [2] proposed a lightweight IoT intrusion detection method that combines deep neural network and bidirectional long short-term memory layers. The DL-BiLSTM model applies IPCA for feature dimensionality reduction and dynamic quantization to reduce the model size. The model achieved 93.13% accuracy and 91.94% F1-score on the CICIOT2023 dataset. However, for static-memory MCU deployment, the sequential BiLSTM architecture remains less suitable. Fares et al. [3] tune the hyperparameters of machine learning and deep learning models on the CICIOT2023 dataset. They selected features and balanced the dataset to reduce the model size. They compared SVM, Random Forest, VGGNet, and DNN models, and the Random Forest model achieved the highest 91.89% accuracy and 92% precision. However, they did not deploy the model on an MCU or embedded XAI. Kuo and Wu [4] propose a framework that combines Decision Tree filtering and a soft-voting ensemble of Random Forest, XGBoost, and AdaBoost. They used the CICIOT2023, CICIOT2024, and Edge-IIoTset datasets to test the model accuracy. The model achieves 99.67% accuracy on CICIOT2023 dataset and 26.66 μ s latency per flow on the Cortex-A53-architected MCU. However the multi-model design targets edge IoT hardware rather than micro-controllers.

Hamdouchi and Idri [6] evaluated TinyML ensemble models using XGBoost and Random Forest models. They used ANOVA, minimum Redundancy, Minimum Relevance (mRMR), and chi-square feature selection methods on the NF-ToN-IoT-v2 and NF-BoT-IoT-v2 datasets. They found the best configuration using the XGBoost with five estimators and 40% selected features. However, this method uses a tree-based architecture and lacks neural TFLite deployment.

Siganos et al. [7] proposed an explainable AI-based IDS using multiple ML and DL models with SHAP explanations. They evaluated the system using the CIC-IoT-Dataset-2022 and IDPS-IEC60870-5-104-Custom datasets. The Random Forest achieved 99.92% accuracy on the CIC-IoT-Dataset-2022 dataset, and XGBoost achieved 85.2% accuracy on IDPS-IEC60870-5-104-Custom dataset. However, SHAP is designed for offline dashboard-based interpretation rather than real-time MCU display. Sharma et al. [10] introduced a self-attention-based explainable IDS using SHAP and LIME methods. They used BoT-IoT, N-BaIoT, and UNSW-NB15 datasets and achieved 99.3%, 99.6%, and 97.9% accuracy, respectively. However, the explanation methods remain much more expensive for direct deployment in the MCU. Adewole et al. [10] proposed an explainable IDS using ensemble learning and rule induction methods and evaluated it on CIC-IDS2017 and CICIOT2023 datasets. XGBoost achieved 98.54% accuracy on CICIOT2023 dataset. However, framework was not designed for MCU deployment.

Chu et al. [9] used the Kolmogorov-Smirnov, Wilcoxon rank-sum, and Mann-Kendall tests to propose a localized EM-NCD for detecting concept drift. The method achieved up of 93.36% average accuracy and 95.21% average F1-score with drift analysis on the Edge-IIoTset dataset. However, without drift analysis, the performance dropped to 64.28% accuracy and a 74.07% F1-score. However, the framework requires multiple statistical detectors and adaptive updates, which makes it too complex for MCU execution.

Overall, existing studies have achieved strong detection accuracy, TinyML feasibility, explainability, and concept drift handling individually. However, they did not combine all aspects of resource-constrained MCU. This leaves a clear gap for an IDS, that provides a small model size, is deployable on MCUs, offers human-readable XAI, and has lightweight drift monitoring.

The summary of the related works is presented in Table I.

III. METHODOLOGY

The visual workflow of the proposed system is presented in Figure 1, which is divided into six main parts: i) dataset acquisition and reduced-task definition, ii) TinyML-oriented feature selection, iii) dataset subset construction and preprocessing, iv) compact neural model construction and deployment conversion, v) human-readable XAI and lightweight drift monitoring, and vi) performance evaluation.

A. Dataset Acquisition and Task Definition

Dataset acquisition and reduced task definition are the first foundational steps in building an effective tinyML-based IDS. In this study, we used the CICIOT2023 dataset, which contains modern IoT traffic and attack patterns. For realistic MCU deployment, the dataset attack class was reduced to four classes: *Benign*, *DDoS*, *DoS*, and *Mirai*. Other classes were removed, which are very rare attack classes, less than 1%. This reduction was intentional and allowed the system to focus on MCU-feasibility.

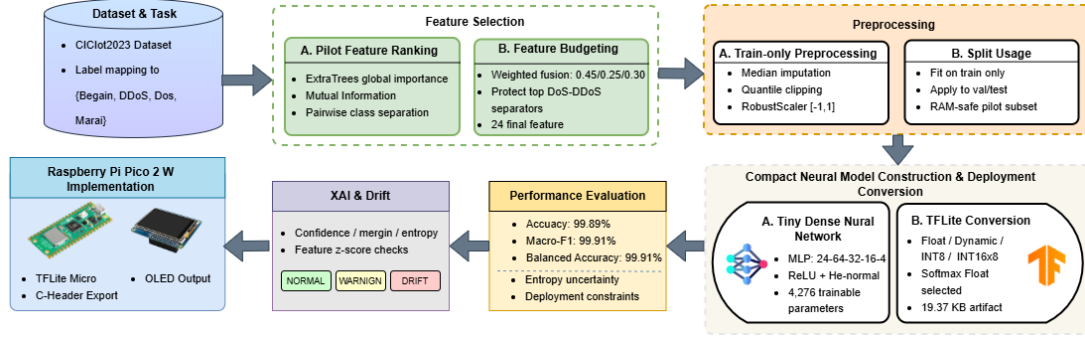


Fig. 1: Workflow of the proposed model

TABLE I: Summary of related studies on IoT intrusion detection

Ref.	Year	Dataset	Method / Algorithm	Results	Limitation
[2]	2023	CICIDS2017, N-BaIoT, CICIoT2023	DL-BiLSTM, IPCA feature reduction, dynamic quantization	Accuracy: 93.13%; F1-score: 91.94%	Sequential model; not MCU-friendly
[3]	2025	CICIoT2023	SVM, Random Forest, VGGNet, DNN	Accuracy: 91.89%; Precision: 92%	No MCU deployment; no XAI
[4]	2026	CICIoT2023, CICIoMT2024, Edge-IIoTset	Decision Tree, Random Forest, XGBoost, AdaBoost, soft voting	Accuracy: 99.67%; Latency: 26.66 μ s	Edge-focused; multi-model design
[7]	2023	CIC-IoT-Dataset-2022, IEC 60870-5-104	ML/DL models with SHAP explanation	Accuracy: 99.99%; F1-score: 99.99%	Offline SHAP; not MCU-ready
[8]	2025	BoT-IoT, N-BaIoT, UNSW-NB15	Self-attention DNN, SHAP, LIME	Accuracy: 99.3%, 99.6%, 97.9%	Heavy XAI; high computation cost
[10]	2025	CIC-IDS2017, CICIoT2023	XGBoost, ensemble learning, rule induction	Accuracy: 98.54%	Not compact neural MCU artifact
[9]	2024	Artificial drift data, Edge-IIoTset	EMNCD, KS test, Wilcoxon test, Mann-Kendall test, WOA-XGBoost	Accuracy: 93.36%; F1-score: 95.21%	Complex drift framework; not MCU-suitable

B. TinyML-Oriented Feature Selection

This subsection outlines the essential steps involved in selecting features to achieve the best possible accuracy and limit the memory size of the model. A pilot subset was first created from the training data. Feature importance was measured using ExtraTrees, mutual information, and pairwise class-separation scores. To separate DoS from DDos, Benign from DDos, Benign from DoS, Mirai from DDos, and Mirai from DoS, extra weight was assigned to these two classes. The three scores were combined into one final score using weights of 0.45, 0.25, and 0.30, respectively. A total of 24 features were selected based on the score.

C. Dataset Subset Construction and Preprocessing

In this phase, we reloaded the dataset using only the final 24 features and introduced balanced class caps to handle the highly imbalanced and large dataset. The final training, validation, and test subsets contain 600,000, 117,519, and 177,709 samples, respectively. For better robustness and conversion stability, all features were clipped to the range of 0.001 to 0.999 of the training set quantile. The features were then normalized using RobustScaler with a 5–95 percentile range. Finally, these features were clipped to the bounded interval $[-1, 1]$. The labels were encoded into four numerical classes. Balanced class weight were computed to reduce bias towards high-volume traffic classes.

D. Compact Neural Model Construction

This model construction and deployment phase is essential for an MCU-friendly model architecture and safe deployment. The proposed classifier model has 24 input features, followed by three hidden layers. The hidden layers contained 64, 32, and 16 neurons. In addition, ReLU activation and He-normal initialization were used in the hidden layers, and the final logit layers produced four output classes. Then, the training setup was stabilized with the Adam optimizer with a learning rate of 10^{-3} and gradient clipnorm of 1.0. Early stopping and learning-rate reduction were used to improve generalization. For MCU deployment, a softmax output layer was added to the trained dense backbone. To test the deployment feasibility, the inference model was converted into Float TFLite, dynamic-range TFLite, full INT8, and INT16x8 formats. Among all the models, the Softmax Float TFLite model was selected. It preserves the accuracy with a 0% drop compared to the FP32 baseline accuracy and also remains compact for MCU deployment.

E. Human-Readable XAI and Lightweight Drift Monitoring

This subsection presents the process of building a human-level trust layer through a simple explanation and monitoring concept drift. For the test samples, the gradient $\times$ input was computed offline. This estimates the importance of the global feature’s importance and supports interpretability. The score

was then normalized into a compact weight vector format and, for MCU compatibility, it was exported as a c-constant. The local explanation is calculated using the following equation:

$$s_i = |x_i \cdot w_i|, \quad (1)$$

Here, x_i is used as a normalized input feature, and w_i is used as a stored feature weight. For drift monitoring, softmax confidence, probability margin, entropy, feature means, and feature standard deviations were used on the validation set. The system checks for low confidence, low margin, high entropy, and out-of-range z -scores for every input. The final concept drift design reported on OLED or serial output as *NORMAL*, *WARNING*, or *DRIFT*.

F. Performance Evaluation

This section presents the parameters used to evaluate the proposed system. Accuracy was used to measure the overall correctness of the predicted classes, as denoted by Eq. (2). The Macro-F1 was used to average the class-wise F1-score, which is denoted by Eq. (3). We also calculated the balanced accuracy, which averages the recall across all classes using Eq. (4). To monitor concept drift, entropy quantifies the prediction uncertainty, as expressed in Eq. (5).

$$\text{Accuracy} = \left(\frac{T_P + T_N}{T_P + T_N + F_P + F_N} \right) \times 100 \quad (2)$$

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C \left(2 \times \frac{P_c \times R_c}{P_c + R_c} \right) \quad (3)$$

$$\text{BAcc} = \frac{1}{C} \sum_{c=1}^C \left(\frac{T_{P_c}}{T_{P_c} + F_{N_c}} \right) \times 100 \quad (4)$$

$$H(\mathbf{p}) = - \sum_{k=1}^C p_k \log(p_k) \quad (5)$$

G. Raspberry Pi Pico 2 W Implementation

This subsection describes the pathway to implement the tinyML model, which was prepared for real MCU execution on a Raspberry Pi Pico 2. From all the deployment-ready models, the Softmax Float TFLite model was selected, and the C header files were converted into `model_data.h`. In addition, the XAI/drift constants were exported into `xai_drift_lcd_constants.h`. TensorFlow Lite Micro was used within the 100kb tensor arena in the Arduino implementation. FullyConnected, ReLU, Softmax, and Reshape were used as the operators. The 24 preprocessed input features were copied into the model input tensor during the inference. The predicted class, confidence, top XAI features, and drift status were displayed using a serial monitor and OLED output.

IV. RESULTS AND DISCUSSIONS

This section presents the model performance under simulated and MCU deployments. This section is subdivided into system configuration, hyperparameters, simulation results, and Raspberry Pi Pico 2 W implementation subsections.

TABLE II: Key training and deployment hyperparameters of the proposed system

Hyperparameter	Value	Hyperparameter	Value
Input features	24	Optimizer	Adam
Initial learning rate	10^{-3}	Batch size	2048
Epochs	60	Clipnorm	1.0
Early stopping patience	8	LR reduction factor	0.5
Min learning rate	10^{-6}	Quantile clipping	0.001 / 0.999
Scaler	Robust (5-95)	Final input range	$[-1, 1]$
Hidden layers	64-32-16	Trainable parameters	4276

TABLE III: Per-class classification performance of the FP32 baseline

Class	Precision	Recall	F1-score
Benign	0.99993	0.99989	0.99991
DDoS	0.99884	0.99772	0.99828
DoS	0.99832	0.99922	0.99877
Mirai	0.99914	0.99938	0.99926

A. System Configuration

A deterministic TensorFlow 2.19.0 environment, NumPy 1.26.4, and fixed seed control were used as the compatible setup. The final TFLite model and XAI/drift constants were prepared for embedded testing on the Raspberry Pi Pico 2W. This MCU uses an RP2350-based board with 520 KB SRAM and supports WIFI wireless communication [11]. The entire research path was developed following the official guidelines of TensorFlow Lite Micro and LiteRT/TFLM [12].

B. Hyperparameters

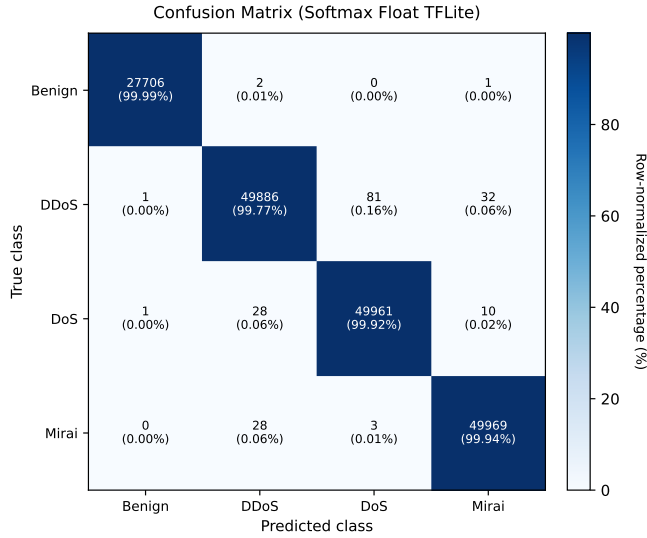
The model was trained using a balanced set of hyperparameters. These settings helped us achieve high accuracy with minimal memory size. Table II provides a summary of the hyperparameters used.

C. Simulation Results

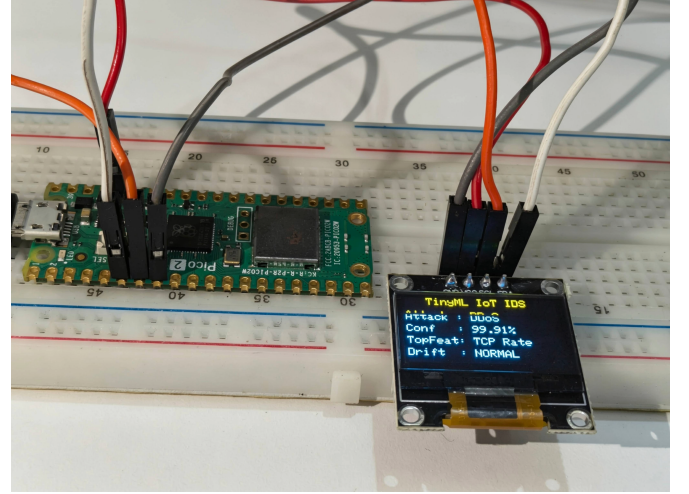
As illustrated in Figure 2(a), the confusion matrix demonstrates an exceptional multiclass classification accuracy. Figure 2(b) validates our physical deployment on a Raspberry Pi Pico 2 W, which successfully classifies real-time traffic. Furthermore, the synchronized training and validation performance curves (Figure 2(c)) and steadily converging loss (Figure 2(d)) collectively confirmed stable learning and robust generalization without overfitting. The multiclass performance of the FP32 baseline is shown in Table III.

Five TensorFlow Lite formats were used to compare the best fits. Among them, the Softmax Float TFLite model achieved exact baseline accuracy with a size of only 19.37 KB. Although others reduced the model size further, they failed to maintain the accuracy. Softmax Float TFLite was selected to maintain prediction quality and drift monitoring. The comparison is presented in Table IV.

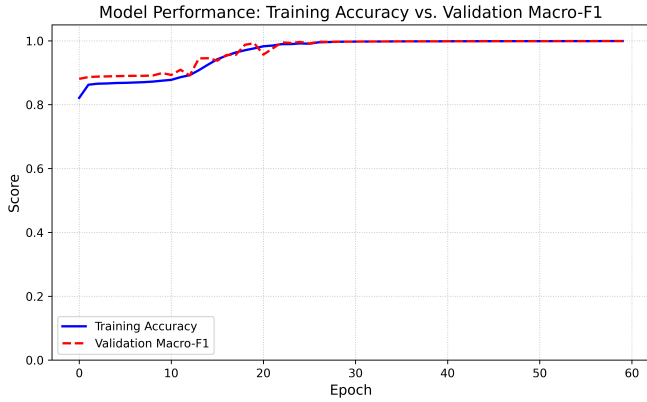
The Softmax Float TFLite reduced the model size by 76% without losing accuracy. This makes it suitable for the deployment of MCU. The embedded XAI module shows important features, such as *TCP traffic* and *UDP traffic* etc. The reduction



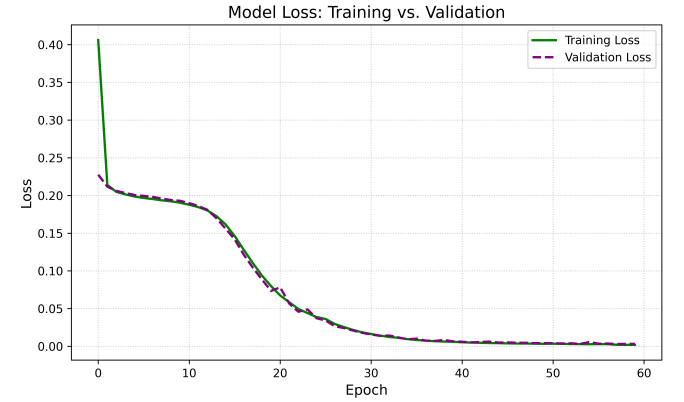
(a)



(b)



(c)



(d)

Fig. 2: Proposed model experimental results: (a) confusion matrix, (b) hardware deployment image, (c) training accuracy and validation macro-F1 curve, and (d) training and validation loss curve.

TABLE IV: Comparison of deployment-ready model formats

Model	Accuracy	Macro-F1	Size (KB)
FP32 Keras	99.89%	99.91%	82.79
Dynamic Range TFLite	87.21%	88.59%	9.98
Full INT8 TFLite	79.90%	82.02%	10.25
INT16x8 TFLite	90.19%	91.28%	11.88
Softmax Float TFLite	99.89%	99.91%	19.37

improves the understanding of prediction. The OLED display shows a human-readable output like attack class, confidence, drift status, and factor for the alert. The drift monitor only reports the drift status as *NORMAL*, *WARNING*, or *DRIFT*. It uses lightweight checks based on confidence, probability margins, entropy, and feature-level distribution shifts.

D. Hardware result

The system was tested on a Raspberry Pi Pico 2 W using the exported TFLite model and XAI/drift header files. The board successfully loaded the model and 24 preprocessed input

features. The OLED displays the attack class label, confidence score, top explanation feature, and drift status. This shows that the system works beyond a simulated environment..

V. LIMITATIONS AND FUTURE WORKS

Despite these encouraging results, certain limitations of this study must be acknowledged. Primarily, the study uses a reduced four-class attack family instead of the full 8 class. Therefore, comparisons with full-class studies should be made cautiously. To make the MCU feasible, the XAI module was intentionally lighter than the full SHAP/LIME analysis. The drift module was designed to detect distribution shifts without online adaptation. Future work may focus on extending the current studies to a full class attack space, advanced quantization-aware training, and linking drift detection with external retraining or federated learning pipelines.

VI. CONCLUSION

This study presents a TinyML-based IoT intrusion detection system. It uses a compact dense neural network and a 24-feature CICIOT2023 dataset. The model was deployed using TensorFlow Lite on a Raspberry Pi Pico 2 W device. The model achieved 99.89% accuracy, reached 99.91% macro-F1 and 99.91% balanced accuracy. The final model size was only 19.37 KB. Among the tested formats, the Softmax Float TFLite model preserved the baseline performance. The deployed system also provided prediction explanations and drift statuses. This improves transparency and reliability. The results showed that IoT intrusion detection can be both accurate and lightweight.

ACKNOWLEDGMENT

The authors express their sincere gratitude to the Ubiquitous, Cloud, and Human-Computer Interaction (UCH) Research Group, Department of Computer Science, American International University-Bangladesh for supporting this research.

REFERENCES

- [1] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIOT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023. [Online]. Available: <https://doi.org/10.3390/s23135941>
- [2] Z. Wang, H. Chen, S. Yang, X. Luo, D. Li, and J. Wang, "A lightweight intrusion detection method for iot based on deep learning and dynamic quantization," *PeerJ Computer Science*, vol. 9, p. e1569, 2023. [Online]. Available: <https://doi.org/10.7717/peerj-cs.1569>
- [3] H. Fares, N. Aknin, G. Lazrek, and M. Zeroual, "Intrusion detection in iot environment using hyperparameters tuned machine and deep learning models on the CICIOT2023 dataset," *Informatica*, vol. 49, no. 5, pp. 325–344, 2025. [Online]. Available: <https://doi.org/10.31449/inf.v49i5.8881>
- [4] C.-W. Kuo and C.-X. Wu, "A lightweight two-stage intrusion detection framework optimized for edge-based IoT environments," *Computers, Materials & Continua*, vol. 87, no. 3, p. 53, 2026. [Online]. Available: <https://doi.org/10.32604/cmc.2026.076767>
- [5] M. Hossain, R. Al Mamun Rudro, R. Razzaque, and K. Nur, "Machine Learning Approaches for Detecting IoT Botnet Attacks: A Comparative Study of N-BaIoT Dataset," in *2024 International Conference on Decision Aid Sciences and Applications (DASA)*, 2024, pp. 1–7. [Online]. Available: <https://doi.org/10.1109/DASA63652.2024.10836622>
- [6] A. Hamdouchi and A. Idri, "Empowering IoT security: Deploying TinyML ensemble techniques for cyberattack detection," *Scientific African*, vol. 29, p. e02809, 2025. [Online]. Available: <https://doi.org/10.1016/j.sciaf.2025.e02809>
- [7] M. Siganos, P. Radoglou-Grammatikis, I. Kotsiuba, E. Markakis, I. Moscholios, S. Goudos, and P. Sarigiannidis, "Explainable AI-based intrusion detection in the Internet of Things," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3600160.3605162>
- [8] K. P. Sharma, T. Nagpal, T. Vora, A. Yadav, M. I. Abdullah, B. Jayaprakash, A. Kashyap, G. Sridevi, A. Bhowmik, and B. B. Bukate, "Interpretable intrusion detection for IoT environments using a self-attention-based explainable AI framework," *Scientific Reports*, vol. 15, p. 39937, 2025. [Online]. Available: <https://doi.org/10.1038/s41598-025-23750-0>
- [9] R. Chu, P. Jin, H. Qiao, and Q. Feng, "Intrusion detection in the IoT data streams using concept drift localization," *AIMS Mathematics*, vol. 9, no. 1, pp. 1535–1561, 2024. [Online]. Available: <https://doi.org/10.3934/math.2024076>
- [10] K. S. Adewole, A. Jacobsson, and P. Davidsson, "Intrusion detection framework for internet of things with rule induction for model explanation," *Sensors*, vol. 25, no. 6, p. 1845, 2025. [Online]. Available: <https://doi.org/10.3390/s25061845>
- [11] Raspberry Pi Ltd., "Raspberry Pi Pico 2 W documentation," Online, 2024. [Online]. Available: <https://www.raspberrypi.com/documentation/microcontrollers/>
- [12] Google, "LiteRT and TensorFlow Lite for microcontrollers documentation," Online, 2025. [Online]. Available: <https://ai.google.dev/edge/litert>